kt. academy

Compose ADVANCED CHEAT SHEET

Modifiers, Theming, Stability & Recomposition



BY MARCIN MOSKALA FROM KT.ACADEMY

Modifiers

Size and Alignment

- `size, width, height` - Narrows constraints to the closest possible value allowed by the previously received constraints. There are also unsafe `requiredSize, requiredWidth,` and `requiredHeight,` which specify size, even if it is outside the parent (but constrained size is reported to the parent).
- `fillMaxWidth, fillMaxHeight, fillMaxSize` - Set constraints to the maximum possible width/height. In practice, a similar result as size with a big value.
- `wrapContentSize` - Allows the content to measure its own size and ignore the minimum size constraints set by the parent.
- `sizeIn, widthIn, heightIn` - Clamps size constraints within a range.
- `defaultMinSize` - Sets the minimum size if no minimum size has been set yet.
- `(Column, Row) weight` - Size the element's height/width proportional to its other weighted sibling elements in the Column/Row. The parent will divide the space remaining after measuring unweighted child elements and distribute it according to this weight.
- `(Box, Column, Row) align` - Align the element horizontally within the parent (horizontally within `Column`, vertically within `Row` , both within `Box`).
- `(Box) matchParentSize` - Makes the size of the element match the size of the `Box` after all other content elements have been measured.
- `layout` - Customizes measurement and placement logic.

Interaction

- `clickable, combinedClickable` - Makes the composable react to user taps, automatically adding accessible ripple indications and semantics. combinedClickable also supports long-click and double-click.
- `scrollable, verticalScroll, horizontalScroll` - Allows the element to scroll its content.
- `draggable` - Enables drag gestures on the UI element.
- `selectable` - Marks an item as part of a selectable group.
- `focusTarget, focusRequester, focusProperties, onFocusChanged` - Fine-grained focus control.
- `semantics, clearAndSetSemantics, testTag, progressSemantics` - Adds semantic properties.
- `indication` - Customizes or disables visual feedback (e.g., ripple).
- `hoverable` - Tracks mouse/pen hover (desktop).
- `pointerInput` - Captures low-level pointer events like drags or gestures (e.g., for custom touch handling).
- `onKeyEvent, onPreviewKeyEvent` - Keyboard handling.
- `keepScreenOn` - for as long as a component with this modifier is visible, the screen will not dim.

Transformations

- `padding` - Enforces space around the component. In practice, if there is enough space in the container, it moves the component; if not, it reduces its constraints.
- `clip` - Limits the space where the composable can be drawn. Everything outside the clip area is not drawn.
- `offset` - Moves the content according to the offset (can be both positive and negative). Offset can move the component outside of the constrained area. In such a case, something might be drawn on top of it.
- `alpha` - Sets the opacity of the content, ranging from 0.0 (transparent) to 1.0 (opaque).
- `scale` - Scales content uniformly or per x/y axis. Can be negative (what flips content).
- `rotate` - Rotates content by the specified angle in degrees (180 is upside-down rotation).
- `graphicsLayer` - Applies GPU-accelerated transformations.

Drawing

- `border` - Draws a border on the component (so down after content).
- `background` - Draws a shape behind the content.
- `drawbehind` - Draw into a Canvas behind the modified content.
- `drawWithContent` - Allows drawing on DrawScope within its lambda before or after the layout's contents. It also allows the modifier to adjust the layout's canvas.
- `drawWithCache` - Allows drawing on DrawScope within its lambda with content that is persisted across draw calls as long as the size of the drawing area is the same or any state objects that are read have not changed. In the event that the drawing area changes, or the underlying state values that are being read change, this method is invoked again to recreate objects to be used during drawing

CompositionLocal

```
data class Greeting(val text: String = "Hello, World!")

val LocalGreeting = compositionLocalOf { Greeting() }

@Composable
private fun Screen() {
    CompositionLocalProvider(LocalGreeting provides Greeting
        ("Good morning!")) {
        A()
    }
}

@Composable
private fun A() {
    B()
}

@Composable
private fun B() {
    val greeting: Greeting = LocalGreeting.current
    Text(greeting.text) // Good morning!
}
```

Prefixed with Local by convention

CompositionLocal definition with default value
(use staticCompositionLocalOf for values that shouldn't change)

Providing CompositionLocal value for a scope and its children

Reading CompositionLocal value

Setting MaterialTheme

1. Define all colors in ui/theme/Color.kt

```
val Purple80 = Color(0xFFD0BCFF)
//...
```

2. Define Typography in ui/theme/Type.kt

```
val Typography = Typography(
    bodyLarge = TextStyle(
        fontSize = 16.sp,
        // ...
    )
    //...
)
```

3. (optional) Define Shapes in ui/theme/Shapes.kt

```
val Shapes = Shapes(
    extraSmall = RoundedCornerShape(size = 4.dp),
    // ...
)
```

4. Define schemes and the MaterialTheme provider in ui/theme/Theme.kt

```
private val DarkColorScheme = darkColorScheme(
    // ...
)

private val LightColorScheme = lightColorScheme(
    // ...
)

@Composable
fun ComposeExercisesTheme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        //Optional, remove it if you don't want to use dynamic colors
        Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else
                dynamicLightColorScheme(context)
        }
        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    MaterialTheme(
        colorScheme =
        colorScheme, typography
        = Typography, shapes =
        Shapes,//Optional
        content = content,
    )
}
```

Important CompositionLocal

Material3 theme locals

All components from the Material3 library choose their colors, text styles, shapes, and effects based on the composition locals we set with the `MaterialTheme` component or with `LocalContentColor` composition local.

MaterialTheme component sets the following composition local: `LocalColorScheme`, `LocalTypography`, `LocalShapes`, `LocalTextSelectionColors`, `LocalTextStyle`, `LocalRippleTheme` (deprecated). Only `LocalIndication` and `LocalTextSelectionColors` are not internal, and we can set them differently.

`LocalContentColor` contains the preferred content color for a given position in the hierarchy. This typically represents the on color for a color in `ColorScheme`. For example, if the background color is `ColorScheme.surface`, this color is typically set to `ColorScheme.onSurface`. In most cases, we don't set it.

We read values from MaterialTheme using MaterialTheme properties:

- `MaterialTheme.colorScheme` - Returns `colorScheme` as specified in `MaterialTheme`.
- `MaterialTheme.typography` - Returns typography as specified in `MaterialTheme`.
- `MaterialTheme.shapes` - Returns shapes as specified in `MaterialTheme`.

Integration/owners

- `LocalLifecycleOwner` - Containing the current `LifecycleOwner`.
- `LocalConfiguration` (Android) - Containing Android device configuration.
- `LocalContext` (Android) - Containing Android `Context`.
- `LocalResources` (Android) - Containing Android `Resource`.
- `LocalView` (Android) - Containing the current Android Compose `View`.
- `LocalOnBackPressedDispatcherOwner` (Android) - Provides an `OnBackPressedDispatcher` that can be used by Composables hosted in an `androidx.activity.ComponentActivity`.
- `LocalViewModelStoreOwner` (Android) - containing the current `ViewModelStoreOwner`.
- `LocalSavedStateRegistryOwner` (Android) - containing the current `SavedStateRegistry`, an interface for plugging components that consume and contribute to the saved state.

Information providers

- `LocalViewConfiguration` - Provides the `ViewConfiguration`, which contains methods for standard constants used in the UI for timeouts, sizes, and distances.
- `LocalWindowInfo` - Provides information about the window that hosts the current Owner.
- `LocalDensity` - Provides the Density to be used to transform DP and SP and pixel units.
- `LocalLayoutDirection` - Provides the layout direction.
- `LocalInspectionMode` - true if the composition is composed inside an `Inspectable` component (i.e. it is a preview)

Additional utils

- `LocalGraphicsContext` (get with `rememberGraphicsLayer()`) - Provides access to a `GraphicsContext` instance for creation of `GraphicsLayers`.
- `LocalAccessibilityManager` - Provides communication with the platform accessibility service.

- `LocalFocusManager` - Can be used to control focus within Compose.
- `LocalAutofillManager` - Can be used to trigger autofill actions. Eg. `AutofillManager.commit`.
- `LocalClipboard` - Provides communication with the platform clipboard service.
- `LocalInputModeManager` - Provides an instance of `InputModeManager`, which controls the current input mode.
- `LocalHapticFeedback` - Provides haptic feedback to the user.
- `LocalSoftwareKeyboardController` - Provides a `SoftwareKeyboardController` that can control the current software keyboard (`null` if the software keyboard cannot be controlled).
- `LocalTextToolbar` - Provides a text-related toolbar.
- `LocalUriHandler` - Provides functionality related to URL, e.g. open URI.
- `LocalTextSelectionColors` - Used to change the `TextSelectionColors` used by text and text field components in the hierarchy.

Stability

Stable	Unstable
Basic types: `Boolean`, `String`, `Int`, `Long`, …	`List`, `Set`, `Map`
`PersistentList`, `PersistentSet`, `PersistentMap`	`LocalDateTime`, `LocalDate`, `LocalTime`
`State`, `MutableState`	`StateFlow`, `LiveData`, …
Function types: `() -> Unit`, `() -> List<Int>`, …	Mutable objects: view models, repositories, …

Compose treats a class as stable if it has an annotation @Stable or @Immutable, or if all its properties are val with stable types. Interfaces are only stable if they have an annotation @Stable or @Immutable, or if they are SAM (in which case they are treated like function types).

```
val classIsStable = hasAnnotation("Stable", "Immutable") ||
        properties.all { it.isVal && it.isStableType }
val interfaceIsStable = hasAnnotation("Stable", "Immutable") ||
        members.singleOrNull()?.isAbstractFunction == true


// Stable ✅
data class UiState(
    val header: String,
    val items: ImmutableList<String>
)
// Unstable ❌
data class UnstableUiState(
    val header: String,
    val items: List<String>
)
// Unstable ❌
data class UnstableUiState2(
    var header: String,
    val items: ImmutableList<String>
)
```

Rule of thumb

- If Compose cannot prove immutability, it assumes instability.
- Use data classes with val properties and stable types
- Prefer persistent collections, avoid mutating collections
- Make copies/new instances to create a new state

Delayed State Reads

Avoid directly reading properties that change frequently as state.

```
@Composable
fun ScrollAwareHeader(
    scrollOffset: Int
)
val listState = rememberLazyListState()
ScrollAwareHeader(
    // ❌ Recomposes the current scope with every value change
    scrollOffset = listState.firstVisibleItemScrollOffset
)
```

Instead, prefer using a lambda to read the value where it is needed:

```
@Composable
fun ScrollAwareHeader(
    scrollOffsetProvider: () -> Int
)

val listState = rememberLazyListState()
ScrollAwareHeader(
    // ✅ Current scope avoids recomposition
    scrollOffsetProvider = { listState.firstVisibleItemScrollOffset }
)
```

Compose conventions

Use CamelCase for naming constants

```
const val DefaultKeyName = "__defaultKey"

val StructurallyEqual: ComparisonPolicy = StructurallyEqualsImpl()

object ReferenceEqual: ComparisonPolicy {
    //...
}

sealed class LoadResult<T> {
    object Loading : LoadResult<Nothing>()
    class Done<T>(val result: T) : LoadResult<T>()
    class Error(val cause: Throwable) : LoadResult<Nothing>()
}

enumclass Status {
    Idle,
    Busy
}
```

Composables should emit x or return value

Component

- UpperCamelCase name
- Emits a single UI root
- Does not return
- Should have a modifier parameter at the first optional position
- Uses this modifier parameter at the top of UI root

Factory function

- lowerCamelCase name
- Does not emit UI
- Returns result
- Uses @Composable only if it needs to remember or use CompositionLocal

```
@Composable
fun FancyButton(
    text: String,
    modifier: Modifier = Modifier
) {
    Text(
        text = text,
        modifier = modifier,
    )
}
```

```
@Composable
fun defaultStyle(): Style = ...

@Composable
fun rememberCoroutineScope():
    CoroutineScope = ...
```

Prefer stateless and controlled components

```
@Composable
fun Checkbox(
    initialValue: Boolean,
    onToggled: (Boolean) -> Unit
) {
    var checkedState by remember { mutableStateOf(initialValue) }
    // ...
}

Checkbox(initialValue = false, onToggled = { callerCheckedState = it})
```

```
@Composable
fun Checkbox(
    value: Boolean,
    onToggled: (Boolean) -> Unit
) {
    // ...
}

Checkbox(callerChecked, onToggled = { callerChecked = it })
```

For complex components, extract a hoistable stable state

```
fun VerticalScrollerState(): VerticalScrollerState =
    VerticalScrollerStateImpl()

@Stable
interface VerticalScrollerState {
    var scrollPosition: Int
    var scrollRange: Int
}
private class VerticalScrollerStateImpl(
    scrollPosition: Int = 0,
    scrollRange: Int = 0
) : VerticalScrollerState {
    private var _scrollPosition by mutableStateOf(scrollPosition)
    override var scrollRange by mutableStateOf(scrollRange)

    override var scrollPosition: Int
        get() = _scrollPosition
        set(value) {
            _scrollPosition = value.coerceIn(0, scrollRange)
        }

    // ...
}
@Composable
fun VerticalScroller(
    verticalScrollerState: VerticalScrollerState =
        remember { VerticalScrollerState() }
) {
    val scrollPosition = verticalScrollerState.scrollPosition
    val scrollRange = verticalScrollerState.scrollRange
}
```

Want to go further?
This cheat sheet captures the core mechanisms behind Compose.
In Advanced Compose course, we are focusing on deep understanding.
The course helps you:

- understand recomposition, layout, and modifiers in practice
- design UI that behaves predictably and is easy to maintain
- learn through lessons, exercises, educational games, and live Q&As

→ Explore Advanced Compose cohort course

